

OPC UA客户端与服务端通信

C# 实现 OPC UA 客户端与服务端通信

简介：OPC UA 是一种跨平台、安全的数据通信标准，广泛应用于工业自动化与物联网领域。

（帮助开发者快速构建 OPC UA 应用程序，并结合西门子 S7-1500 PLC 进行实际通信测试，提升工业自动化系统的兼容性与通信效率。）

1. OPC UA 通信协议概述

OPC UA 是一种面向服务的通讯协议，专为工业自动化系统设计，支持跨平台、安全、可靠的数据交换。其核心优势在于统一了原有 OPC Classic 规范，消除了 COM/DCOM 的依赖，适用于现代网络环境。

1.1 OPC UA 的基本概念

OPC UA 不仅是一种通讯协议，更是一个完整的数据模型和信息服务框架。它通过标准接口实现不同厂商设备之间的互操作性，支持复杂的数据结构、方法调用、报警与事件机制等高级功能。

- 跨平台性**：基于 TCP/IP 协议栈，支持多种操作系统（如 Windows、Linux、RTOS 等）。
- 安全性**：内置加密、身份认证、访问控制等安全机制。
- 可扩展性**：支持自定义数据类型和服务接口，便于系统集成与扩展。

1.2 OPC UA 的架构组成

OPC UA 采用分层架构设计，主要包括以下几个层级：

层级	名称	作用
1	应用层	提供客户端与服务端的应用接口（如读写、订阅等）
2	服务层	定义操作语义（如读、写、订阅、方法调用等）
3	会话层	管理会话建立、身份验证与安全策略
4	传输层	支持多种传输协议（如 SOAP、HTTPS、二进制协议）
5	网络层	基于 TCP/IP 进行数据传输

此外，OPC UA 还定义了**地址空间模型**，将设备、变量、方法等资源抽象为节点（Node），并通过节点 ID 进行访问。

1.3 OPC UA 的通讯模型

OPC UA 通信模型基于客户端 - 服务端架构（CS），支持以下核心通讯方式：

- **请求 - 响应模式**：客户端发送请求，服务端返回结果（如读写操作）。
- **发布 - 订阅模式**：支持实时数据变化通知（用于高效的数据更新）。
- **方法调用**：客户端可调用服务端定义的方法。

这种灵活的通信机制使得 OPC UA 不仅适用于传统的 PLC、SCADA 系统，也能很好地集成到工业物联网（IIoT）平台中。

读写方式

无论是 OPC DA 还是 OPC UA，在读写方式方面都是类似的。OPC 读写方式主要有三种，分别是同步方式、异步方式及订阅方式，订阅方式仅针对读取有效。

1. **同步方式**：同步方式是指当客户端发送请求后，必须等到服务器响应全部完成后才能返回，期间将一直处于等待状态，因此当多客户端向服务器操作时，客户端程序产生阻塞，同步通讯适用于客户端较少，数据量较小的场合，同步方式的工作流程如下图所示：![img](client server Call myGroup.SynchRead () Reply 公众号 · 上位机 Guide)
2. **异步方式**：异步方式是指当客户端发送请求后立即返回，不需要等待服务器的响应，可以进行其他操作，当服务器完成响应后会自动通知客户端，因此相对于同步通讯，异步通讯的效率更高，异步方式的工作流程如下图所示：![img](client server Call myGroup.ASynchRead () myGroup_AsyncReadComplete () Reply 公众号 · 上位机 Guide)
3. **订阅方式**：订阅方式在初始化时，就需要订阅相应的通信组 Group，这样当服务器的 Group 组内有数据发生变化时，就会自动刷新客户端数据，客户端只需要向服务器发送一次请求，因此订阅方式的效率是非常高的，订阅方式的工作流程如下图所示：![img](client server Subscribe myGroup.IsSubscribed Notify myGroup_DataChange () Notify myGroup_DataChange () 公众号 · 上位机 Guide)

如果要实现 OPC 通信，首先需要搭建一个 OPC 服务器环境，这里我们采用比较主流的 KepServer 软件，硬件采用西门子 S7-1200PLC，通过 KepServer 对接 S7-1200，然后作为服务器，可以通过 OPCDA 或 OPCUA 进行访问。

1.4 OPC UA 在工业控制系统中的应用价值

随着工业 4.0 和智能制造的发展，OPC UA 因其开放性、标准化和高安全性，已成为工业通信的首选协议。它不仅用于设备间的数据交换，还广泛应用于：

- 工厂级数据集成
- 设备状态监测与预测性维护
- 云边协同架构中的数据上传
- 跨厂商系统互联

掌握 OPC UA 的通信机制，是构建现代化工业自动化系统的基石。

2. C# 实现 OPC UA 客户端连接配置

在工业自动化系统中，OPC UA 客户端是与服务端进行数据交互的核心组件。使用 C# 语言实现 OPC UA 客户端连接配置，不仅能充分发挥 .NET 平台的开发优势，还能借助 OPC Foundation 提供的标准 SDK，快速构建高效、稳定的通信程序。深入探讨 OPC UA 客户端连接的实现机制，涵盖从通信模型、SDK 集成、连接建立到异常处理的完整流程。

2.1 OPC UA 客户端连接原理

OPC UA 客户端与服务端之间的连接建立是基于面向服务的架构（SOA）设计的，其通信模型以会话（Session）为核心，通过端点（Endpoint）和安全策略保障通信的安全性和可靠性。理解这些基本原理是实现客户端连接的前提。

2.1.1 客户端通信模型与会话机制

OPC UA 客户端与服务端之间的通信采用客户端 - 服务端模型，主要通过以下四个步骤完成：

- 1. 发现服务端：客户端通过查找服务端的端点信息（如 IP 地址和端口）建立初始连接。
- 2. 建立安全通道：在传输层之上建立加密通信通道，确保数据传输的安全性。
- 3. 创建会话：客户端向服务端发起会话请求，服务端验证身份后创建会话对象。
- 4. 数据交互：在会话生命周期内，客户端可以调用服务端提供的各种服务（如读写、订阅等）。

会话机制确保了通信的稳定性和身份验证的持续性。每个会话都有一个唯一的 Session ID，并支持会话的续订和关闭操作。

会话建立流程图：

找不到“Pasted image 20251014162332.png”。

2.1.2 端点选择与安全策略配置

OPC UA 支持多种安全策略，包括 None、Sign、SignAndEncrypt 等，客户端在连接时需要选择合适的策略和服务端匹配。端点（Endpoint）是客户端连接服务端的关键信息，包括协议、地址、端口、安全策略等。

端点选择流程：

- 1. 客户端获取服务端的所有可用端点。
- 2. 根据客户端配置的安全策略筛选出匹配的端点。
- 3. 选择最优端点并建立连接。

以下是一个端点信息的表格实例：

端点 URL	安全策略	安全模式	证书指纹
opc.tcp://192.168.1.10:4840	Basic256Sha256	SignAndEncrypt	A1:B2:C3:D4:E5:F6:G7:H8

端点 URL	安全策略	安全模式	证书指纹
opc.tcp://192.168.1.10:4840	None	None	-

客户端应优先选择加密强度高、安全性强的端点进行连接，避免使用 **None** 策略，以防止通信过程中的数据泄露。

2.2 使用 C# 建立 OPC UA 客户端连接

C# 语言在工业自动化领域具有广泛的应用，结合 OPC Foundation 提供的 .NET Standard SDK（如 OPCFoundation.NETStandard.Opc.Ua），开发者可以快速构建 OPC UA 客户端程序。

2.2.1 引入 OPC UA SDK 库（如 OPCFoundation.NETStandard.Opc.Ua）

首先，需在项目中引入 OPC UA SDK 库。可以通过 NuGet 包管理器安装：

bash

```
dotnet add package OPCFoundation.NETStandard.Opc.Ua
```

引入后，可以在代码中使用命名空间：

csharp

```
using Opc.Ua;  
using Opc.Ua.Client;
```

2.2.2 创建客户端实例并连接至服务端

创建客户端实例并连接服务端的步骤如下：

1. 初始化客户端信息
2. 获取服务端端点信息
3. 创建客户端实例
4. 建立连接

```
// 1. 配置客户端应用描述  
var applicationDescription = new ApplicationDescription  
{  
    ApplicationName = "MyOPCClient",  
    ApplicationType = ApplicationType.Client  
};
```

```
// 2. 创建配置对象
var configuration = new Configuration
{
    Application = applicationDescription
};

// 3. 获取服务端端点
var endpointUrl = "opc.tcp://192.168.1.10:4840";
var endpoints = CoreClientUtils.GetEndpoints(endpointUrl, false).Result;

// 4. 选择安全策略匹配的端点
var selectedEndpoint = endpoints.Find(e => e.SecurityPolicyUri ==
SecurityPolicies.Basic256Sha256);

// 5. 创建客户端实例
var client = new SessionClient(
    configuration,
    selectedEndpoint,
    new UserIdentity(),
    SessionChannel.DefaultOperationTimeout
);

// 6. 建立会话
await client.ConnectAsync();
Console.WriteLine("连接成功!");
```

该代码片段使用了 **UserNameIdentityToken**，将用户名和密码封装为身份验证令牌。服务端会验证该令牌后决定是否允许建立会话。

2.3 客户端连接异常处理与重连机制

在实际工业环境中，网络不稳定、服务端宕机等情况可能导致连接中断。因此，客户端应具备完善的异常处理和自动重连机制。

2.3.1 连接失败的常见原因分析

连接失败可能由以下原因引起：

原因类型	描述
网络不通	IP 地址错误或端口未开放
安全策略不匹配	客户端与服务端安全策略不一致
身份验证失败	用户名或密码错误
服务端未启动	OPC UA 服务端未运行

原因类型	描述
证书问题	证书未安装或证书链不完整

在开发中，应根据具体的异常类型进行日志记录和提示。

2.3.2 实现自动重连与异常日志记录

自动重连机制可通过定时尝试连接或监听会话状态变化实现。以下是一个简单的自动重连逻辑示例：

csharp

```
private async Task ReconnectAsync(SessionClient client, string endpointUrl)
{
    int retryCount = 0;
    const int maxRetries = 5;

    while (retryCount < maxRetries)
    {
        try
        {
            await Task.Delay(5000); // 每5秒重试一次
            await client.ReconnectAsync(endpointUrl,
10000).ConfigureAwait(false);
            Console.WriteLine("重连成功! ");
            break;
        }
        catch (Exception ex)
        {
            retryCount++;
            Console.WriteLine($"第{retryCount}次重连失败: {ex.Message}");
            LogError(ex); // 记录异常日志
        }
    }

    if (retryCount >= maxRetries)
    {
        Console.WriteLine("达到最大重试次数，连接失败。");
    }
}
```

代码逻辑说明：

- ReconnectAsync：尝试重新连接服务端。

- `Task.Delay(5000)`：设置重连间隔时间为 5 秒。
- `LogError`：将异常信息写入日志文件，便于后续排查。

日志记录示例函数：

csharp

```
private void LogError(Exception ex)
{
    string logMessage = $"[{DateTime.Now}] 异常信息: {ex.Message}\n堆栈:
{ex.StackTrace}\n";
    File.AppendAllText("opc_client_error.log", logMessage);
}
```

此函数将异常信息追加写入日志文件 `opc_client_error.log`，便于后期分析和调试。

本章从 OPC UA 客户端连接的基本原理出发，详细讲解了通信模型、会话机制、端点选择、C# SDK 的使用以及连接建立的具体步骤。并通过代码示例展示了如何在 C# 中实现客户端连接、身份验证、异常处理与自动重连机制，为后续客户端数据与订阅机制的实现打下了坚实基础。

3. C# 实现 OPC UA 客户端数据读写操作

在工业自动化系统中，OPC UA 协议提供了高效、安全的数据访问机制。客户端通过 OPC UA 协议可以访问服务端节点的地址空间，实现对节点数据的读取和写入操作。本章将详细介绍如何使用 C# 语言实现 OPC UA 客户端对节点数据的读写功能，包括同步与异步数据读取方式、写入操作流程、以及数据验证机制。我们将基于 `OPCFoundation.NETStandard.Opc.Ua` 库进行开发，并结合代码示例详细解析每一步操作。

3.1 OPC UA 节点与地址空间访问机制

OPC UA 的地址空间由多个节点组成，每个节点都有唯一的节点 ID (`NodeId`)，并通过节点属性（如 `Value`，`DisplayName`，`Description`）提供访问接口。客户端通过调用服务端的数据访问服务来获取或修改节点的值。

3.1.1 节点 ID 与节点属性解析

在 OPC UA 中，节点是地址空间的基本组成单位。每个节点都有一个唯一的 `NodeId`，用于标识该节点。`NodeId` 由命名空间索引和标识符组成，例如：

csharp

```
NodeId nodeId = new NodeId("ns=2;s=Demo.Static.Scalar.Int32", context);
```

在上述代码中：

- `ns=2` 表示命名空间索引为 2。
- `s=Demo.Static.Scalar.Int32` 表示节点的字符串标识符。

每个节点具有多个属性，其中最常用的是：

- **Value**：节点的当前值。
- **DisplayName**：节点的显示名称。
- **Description**：节点的描述信息。
- **DataType**：节点值的数据类型。
- **ValueRank**：数组维度信息。

属性名称	描述
Value	节点的当前数值
DisplayName	显示名称，通常用于 UI 展示
Description	描述信息，用于说明节点用途
DataType	数据类型，如 Int32、String 等
ValueRank	数组维度，表示是否为数组

节点的属性可以通过 `ReadValueId` 结构进行访问：

csharp

```
ReadValueId readValueId = new ReadValueId
{
    NodeId = nodeId,
    AttributeId = Attributes.Value
};
```

其中，`AttributeId` 用于指定要读取的属性。OPC UA 定义了一系列常量表示不同的属性，如：

- `Attributes.Value`：节点的值。
- `Attributes.DisplayName`：节点的显示名称。
- `Attributes.Description`：节点的描述信息。

3.1.2 数据访问服务的调用流程

OPC UA 客户端通过调用服务端的 `Read` 和 `Write` 服务来实现数据的读写操作，基本流程如下：

读取操作：

写入操作：

在 C# 中，这些操作通过 `Session` 对象完成。`Session` 对象是在客户端连接服务端后创建的会话实例，它提供了服务端数据的访问能力。

3.2 使用 C# 进行数据读取操作

OPC UA 客户端支持同步和异步两种方式读取数据。同步方式适用于简单、实时性要求不高的场景，而异步方式适用于需要高性能和并发处理的场景。

3.2.1 同步读取单个 / 多个节点数据

同步读取是最直接的方式，适用于少量节点的读取操作，以下是一个简单读取单个节点值的例子：

csharp

```
// 创建ReadValueId对象
ReadValueId readValueId = new ReadValueId
{
    NodeId = new NodeId("ns=2;s=Demo.Static.Scalar.Int32", context),
    AttributeId = Attributes.Value
};

// 构建读取请求
ReadRequest readRequest = new ReadRequest
{
    NodesToRead = new[] { readValueId },
    MaxAge = 0,
    TimestampsToReturn = TimestampsToReturn.Both
};

// 调用Read方法
ReadResponse readResponse = session.Read(null, readRequest);

// 解析返回值
DataValue dataValue = readResponse.Results[0];
Console.WriteLine("节点值: " + dataValue.Value);
```

代码说明：

- `readValueId`：定义要读取的节点及其属性。
- `MaxAge`：允许服务端返回缓存数据的最大时间（毫秒），设为 0 表示请求实时数据。
- `TimestampsToReturn`：指定返回的时间戳类型，`Both` 表示同时返回服务器时间和源时间。
- `session.Read()`：调用 OPC UA 服务端的 Read 服务，返回结果数组。

- `dataValue.Value`：获取节点的值。

若需读取多个节点，只需在 `NodesToRead` 数组中添加多个 `ReadValueId` 对象即可：

csharp

```
ReadValueId[] nodesToRead = new ReadValueId[]
{
    new ReadValueId { NodeId = new NodeId("ns=2;s=Demo.Static.Scalar.Int32",
context), AttributeId = Attributes.Value },
    new ReadValueId { NodeId = new NodeId("ns=2;s=Demo.Static.Scalar.String",
context), AttributeId = Attributes.Value }
};

ReadRequest readRequest = new ReadRequest
{
    NodesToRead = nodesToRead,
    MaxAge = 0,
    TimestampsToReturn = TimestampsToReturn.Both
};

ReadResponse readResponse = session.Read(null, readRequest);

foreach (DataValue value in readResponse.Results)
{
    Console.WriteLine("节点值: " + value.Value);
}
```

3.2.2 异步读取数据并处理回调

异步读取适用于需要并发处理多个请求的场景。通过使用 `BeginRead` 和 `EndRead` 方法实现异步调用：

csharp

```
// 定义回调函数
private static void ReadCompleted(IAsyncResult ar)
{
    try
    {
        ReadResponse readResponse = session.EndRead(ar);
        foreach (DataValue value in readResponse.Results)
        {
            Console.WriteLine("异步读取值: " + value.Value);
        }
    }
}
```

```

    }
    catch (Exception ex)
    {
        Console.WriteLine("异步读取异常: " + ex.Message);
    }
}

// 异步调用Read方法
session.BeginRead(null, readRequest, ReadCompleted, null);

```

代码说明：

- BeginRead：启动异步读取操作。
- ReadCompleted：异步回调函数，在读取完成后自动调用。
- EndRead：在回调函数中获取异步操作的结果。

异步方式可以显著提高客户端的响应性能，尤其是在处理大量节点读取任务时，推荐使用异步模式。

3.3 使用 C# 进行数据写入操作

OPC UA 客户端不仅可以读取节点数据，还可以向节点写入新值。写入操作需要构造 WriteValue 对象，并调用 Write 方法发送请求。

3.3.1 写入节点属性与值的基本流程

写入单个节点值的示例：

csharp

```

// 定义要写入的节点
NodeId nodeId = new NodeId("ns=2;s=Demo.Static.Scalar.Int32", context);

// 创建WriteValue对象
WriteValue writeValue = new WriteValue
{
    NodeId = nodeId,
    AttributeId = Attributes.Value,
    Value = new DataValue { Value = 12345 }
};

// 构建写入请求
WriteRequest writeRequest = new WriteRequest
{
    NodesToWrite = new[] { writeValue }
}

```

```
};

// 调用Write方法
WriteResponse writeResponse = session.Write(null, writeRequest);

// 检查写入结果
if (writeResponse.Results[0].Succeeded())
{
    Console.WriteLine("写入成功! ");
}
else
{
    Console.WriteLine("写入失败: " + writeResponse.Results[0].ToString());
}
```

代码说明：

- WriteValue：封装了节点 ID、属性 ID 和写入的值。
- DataValue：包含实际写入的数据值。
- session.Write()：调用 OPC UA 服务端的 Write 服务。
- Results[0].Succeeded()：检查写入操作是否成功。

3.3.2 数据写入结果的验证与反馈机制

写入操作完成后，客户端应验证服务端的响应，确保数据成功写入。可以通过检查 WriteResponse.Results 数组中的状态码来判断写入是否成功：

csharp

```
StatusCode status = writeResponse.Results[0];
if (status.Code == StatusCodes.Good)
{
    Console.WriteLine("数据写入成功! ");
}
else
{
    Console.WriteLine("数据写入失败，错误码: " + status.Code);
}
```

此外，可以结合读取操作再次读取该节点的值，进行对比验证：

csharp

```
// 读取刚写入的节点值
ReadValueId readValueId = new ReadValueId
{
    NodeId = nodeId,
    AttributeId = Attributes.Value
};

ReadResponse readResponse = session.Read(null, new ReadRequest { NodesToRead =
new[] { readValueId } });
DataValue readValue = readResponse.Results[0];

if (readValue.Value.Equals(writeValue.Value.Value))
{
    Console.WriteLine("写入值验证通过!");
}
else
{
    Console.WriteLine("写入值不一致, 请检查服务端配置!");
}
```

此验证机制可有效防止因服务端处理异常或网络问题导致的数据不一致情况，提高系统的健壮性。

4. C# 实现 OPC UA 客户端订阅机制

在工业自动化系统中，实时数据监控是关键需求之一。OPC UA 提供了强大的订阅机制，使得客户端能够高效地接收服务端数据的变化通知。本章将深入探讨 OPC UA 的订阅机制原理，并通过 C# 实现客户端的订阅管理与数据变化回调逻辑，帮助开发者构建高性能的实时监控系統。

4.1 OPC UA 订阅机制原理

OPC UA 的订阅机制是实现客户端实时获取服务端数据变化的核心手段。通过订阅模型，客户端可以避免频繁轮询带来的性能损耗，提升系统响应效率。

4.1.1 数据变化通知机制与订阅模型

OPC UA 定义了三种数据变化触发类型（DataChangeTrigger）来控制数据变化通知的行为：

触发类型	描述
Status	仅当节点值的状态发生变化时触发通知
Status, Value	节点值的状态或数值发生变化时触发通知
Status, Value, Timestamp	节点值的状态、数值或时间戳发生变化时触发通知

订阅模型主要由以下几个组件构成：

- **Subscription**：代表一个订阅实例，包含采样间隔、优先级、队列大小等配置。
- **MonitoredItem**：表示客户端监视的节点项，每个监视项对应一个 OPC UA 节点。
- **NotificationMessage**：当数据变化时，服务端会通过此消息将变化的数据发送给客户端。

订阅流程图：

4.1.2 订阅服务的生命周期管理

订阅服务的生命周期包括创建、运行、更新和删除几个阶段：

1. **创建阶段**：客户端调用 `CreateSubscription` 服务创建订阅对象，并指定采样间隔、优先级等参数。
2. **运行阶段**：客户端添加监视项，服务端开始监控节点数据变化。
3. **更新阶段**：客户端可通过 `ModifySubscription` 和 `SetMonitoringMode` 接口调整订阅参数或监视项状态。
4. **删除阶段**：调用 `DeleteSubscription` 删除订阅，释放资源。

创建订阅的基本调用：

csharp

```
// 创建订阅对象
var request = new CreateSubscriptionRequest
{
    RequestHeader = new RequestHeader { AuthenticationToken =
session.AuthenticationToken },
    ClientHandle = 123456,
    RequestedPublishingInterval = 1000, // 每秒发布一次
    RequestedLifetimeCount = 60,        // 生命周期内最大通知次数
    RequestedMaxKeepAliveCount = 10,    // 心跳间隔
    MaxNotificationsPerPublish = 100,   // 每次发布的最大通知数
    PublishingEnabled = true,           // 启用发布
    Priority = 1                        // 优先级
};

// 发送请求
var response = await session.CreateSubscriptionAsync(request,
CancellationToken.None);
uint subscriptionId = response.SubscriptionId;
```

逐行分析：

- RequestedPublishingInterval：设置客户端期望的数据更新频率。
- ClientHandle：是客户端自定义的标识符，用于后续匹配订阅。
- PublishingEnabled：决定是否立即开始接收通知。
- CreateSubscriptionAsync：异步调用创建订阅，返回 SubscriptionId 供后续操作使用。

4.2 创建与管理订阅客户端

在实际开发中，开发者需要能够动态地创建订阅对象，配置采样间隔，添加监控项并设置数据变化的触发条件。

4.2.1 创建订阅对象与定义采样间隔

采样间隔（Sampling Interval）决定了服务端对节点值的采样频率。如果节点值在两个采样周期之间发生变化，服务端可能会忽略该变化，除非设置了“死区”(Deadband) 机制。

完整的订阅对象示例：

csharp

```
// 创建订阅对象
var createSubscriptionRequest = new CreateSubscriptionRequest
{
    RequestHeader = new RequestHeader { AuthenticationToken =
session.AuthenticationToken },
    RequestedPublishingInterval = 1000, // 每秒采样一次
    RequestedLifetimeCount = 60,
    RequestedMaxKeepAliveCount = 10,
    MaxNotificationsPerPublish = 100,
    PublishingEnabled = true,
    Priority = 1
};

var createSubscriptionResponse = await
session.CreateSubscriptionAsync(createSubscriptionRequest,
CancellationToken.None);
uint subscriptionId = createSubscriptionResponse.SubscriptionId;
```

参数说明：

参数	描述
RequestedPublishingInterval	客户端请求的发布间隔（毫秒）
RequestedLifetimeCount	在生命周期内允许的最大通知数量
RequestedMaxKeepAliveCount	两次数据变化之间允许的最大心跳间隔数

参数	描述
MaxNotificationsPerPublish	每次发布操作的最大通知条目数
PublishingEnabled	是否启用订阅发布机制

4.2.2 添加监视项并设置触发条件

添加监视项的过程包括指定节点、设置采样间隔、触发条件和死区值。以下是一个添加监视项的完整示例：

csharp

```
// 创建监视项
var monitoredItem = new MonitoredItemCreateRequest
{
    ItemToMonitor = new ReadValueId
    {
        NodeId = new NodeId("ns=2;s=MyVariable"), // 监视的节点ID
        AttributeId = Attributes.Value              // 监视值属性
    },
    MonitoringMode = MonitoringMode.Reporting,      // 报告模式
    RequestedParameters = new MonitoringParameters
    {
        ClientHandle = 987654,
        SamplingInterval = 1000,                    // 每秒采样一次
        Filter = new DataChangeTrigger(DataChangeTrigger.Status |
DataChangeTrigger.Value),
        QueueSize = 10,                             // 队列大小
        DiscardOldest = true                         // 丢弃旧数据
    }
};

var monitoredItems = new[] { monitoredItem };
var createMonitoredItemsRequest = new CreateMonitoredItemsRequest
{
    RequestHeader = new RequestHeader { AuthenticationToken =
session.AuthenticationToken },
    SubscriptionId = subscriptionId,
    TimestampsToReturn = TimestampsToReturn.Both,
    ItemsToCreate = monitoredItems
};

var createMonitoredItemsResponse = await
session.CreateMonitoredItemsAsync(createMonitoredItemsRequest,
CancellationToken.None);
```


参数说明：

参数	描述
ItemToMonitor	指定监视的节点及其属性
MonitoringMode	监视模式（Reporting、Sampling、Disabled）
SamplingInterval	采样间隔，单位毫秒
Filter	数据变化触发条件
QueueSize	通知队列大小，用于缓存未发送的通知
DiscardOldest	队列满时是否丢弃旧数据

4.3 数据变化事件处理与回调函数

当服务端节点数据发生变化时，客户端会通过订阅机制接收到通知。为了处理这些通知，开发者需要实现回调函数来解析并处理数据变化事件。

4.3.1 接收实时数据变化通知

订阅机制的核心在于客户端能够异步接收来自服务端的通知。OPC UA 使用 Publish 服务来发送这些通知，以下是一个处理 Publish 响应的示例：

csharp

```
// 循环监听 Publish 响应
while (true)
{
    var publishRequest = new PublishRequest
    {
        RequestHeader = new RequestHeader { AuthenticationToken =
session.AuthenticationToken }
    };

    var publishResponse = await session.PublishAsync(publishRequest,
CancellationToken.None);

    foreach (var notification in
publishResponse.NotificationMessage.NotificationData)
    {
        if (notification is DataChangeNotification dataChange)
        {
            foreach (var item in dataChange.MonitoredItems)
            {
                Console.WriteLine($"ClientHandle: {item.ClientHandle}, Value:
```

```
{item.Value.Value}, Timestamp: {item.Value.SourceTimestamp}");
    }
}
}
```

分析：

- PublishAsync：是客户端主动请求通知的方式。
- DataChangeNotification：表示数据变化的通知。
- MonitoredItems：是一组发生变化的节点项。
- SourceTimestamp：表示服务端记录的值变化时间。

4.3.2 在 C# 中实现数据变化回调逻辑

为了实现更灵活的数据处理逻辑，开发者通常会注册一个回调函数在数据变化时自动触发，以下是使用事件注册方式的示例：

csharp

```
// 注册回调函数
session.Notification += OnNotificationReceived;

private void OnNotificationReceived(Session session, NotificationEventArgs e)
{
    foreach (var notification in e.NotificationMessage.NotificationData)
    {
        if (notification is DataChangeNotification dataChange)
        {
            foreach (var item in dataChange.MonitoredItems)
            {
                Console.WriteLine($"Node: {item.MonitoredItemId}, Value:
{item.Value.Value}, Status: {item.Value.StatusCode}");
            }
        }
    }
}
```

逻辑说明：

- Notification：是 Session 类的一个事件，每当接收到通知时会触发。
- OnNotificationReceived：是回调方法，用于处理通知数据。
- 可以在回调中加入日志记录、数据持久化、UI 更新等操作。

5. OPC UA 服务端节点定义与创建

OPC UA 的核心在于地址空间的结构化管理，服务端通过定义和创建节点，将实际的设备、变量、方法等信息以统一的方式暴露给客户端，实现跨平台、标准化的数据交互。深入讲解如何使用 C# 在 OPC UA 服务端定义和创建自定义节点，包括节点层级结构的设计、属性配置、数据绑定等关键内容，帮助开发者构建一个结构清晰、可扩展的服务端地址空间。

5.1 OPC UA 服务端节点的基本概念

5.1.1 节点与地址空间的关系

OPC UA 服务端的地址空间是由一组节点（Node）构成的树状结构。每个节点代表一个对象、变量、方法或属性，并通过节点 ID（NodeId）进行唯一标识。

节点类型	描述
Object	用于表示设备、模块或功能单元
Variable	表示具有值的变量节点，如传感器数值
Method	可调用的操作，如“重启设备”
Property	对象或变量的附加属性，如描述、单位
ObjectType	定义对象的模板，用于实例化
VariableType	定义变量的模板，用于实例化

5.1.2 节点属性详解

每个节点都具有多个标准属性，常见的包括：

- NodeId**：节点的唯一标识符。
- NodeClass**：节点的类型（如 Object、Variable）。
- BrowseName**：用于浏览的节点名称。
- DisplayName**：显示给用户的名称。
- Description**：节点的描述信息。
- Value**：仅用于变量节点，表示当前值。
- DataType**：变量节点的数据类型（如 Int32、String）。
- ValueRank**：数据维度（-1 表示单值，1 表示一维数组）。

5.2 使用 C# 创建 OPC UA 服务端节点

5.2.1 准备开发环境与引用库

要使用 C# 开发 OPC UA 服务端节点，首先需要引入 OPC UA SDK。推荐使用开源的 OPCFoundation.NETStandard.Opc.Ua 库。

bash

```
// 安装NuGet包
dotnet add package OPCFoundation.NETStandard.Opc.Ua
```

csharp

```
// 引用命名空间
using Opc.Ua;
using Opc.Ua.Server;
```

5.2.2 创建自定义对象与变量节点

如何在 OPC UA 服务端创建一个自定义对象节点以及其下属的变量节点：

csharp

```
public class MyCustomNodeManager : CustomNodeManager2
{
    public MyCustomNodeManager(IServerInternal server,
        ApplicationConfiguration configuration)
        : base(server, configuration)
    {
    }

    protected override NodeId NewAddressSpaceRoot()
    {
        return ObjectIds.ObjectsFolder;
    }

    public override void CreateAddressSpace(IDictionary<NodeId, INode>
externalReferences)
    {
        base.CreateAddressSpace(externalReferences);

        // 创建自定义对象
        var myObject = new BaseObjectState(null)
        {
            NodeId = new NodeId("MyCustomObject", NamespaceIndex),
            BrowseName = new QualifiedName("MyCustomObject", NamespaceIndex),
            DisplayName = new LocalizedText("MyCustomObject"),
        }
    }
}
```

```

        Description = new LocalizedText("This is a custom object
representing a device.")
    };
    AddReference(ObjectIds.ObjectsFolder, ReferenceTypeIds.Organizes,
false, myObject.NodeId);
    externalReferences.Add(myObject.NodeId, myObject);

    // 创建变量节点
    var temperatureVariable = new BaseDataVariableState(myObject)
    {
        NodeId = new NodeId("Temperature", NamespaceIndex),
        BrowseName = new QualifiedName("Temperature", NamespaceIndex),
        DisplayName = new LocalizedText("Temperature"),
        Description = new LocalizedText("Current temperature value of the
device."),
        DataType = DataTypeIds.Double,
        ValueRank = ValueRanks.Scalar,
        Value = 25.0
    };
    myObject.AddReference(ReferenceTypeIds.HasComponent, false,
temperatureVariable.NodeId);
    externalReferences.Add(temperatureVariable.NodeId,
temperatureVariable);
    }
}

```

代码逐行解读：

1. 第 1-5 行：定义一个自定义的节点管理器类，继承自 CustomNodeManager2。
2. 第 7-9 行：构造函数，调用基类构造方法。
3. 第 11-13 行：设置地址空间的根节点为标准的 ObjectsFolder。
4. 第 15-33 行：重写 CreateAddressSpace 方法，在服务端地址空间中创建自定义对象和变量节点。
5. 第 17-22 行：创建一个 BaseObjectState 类型的对象节点，表示设备对象。
6. 第 23-24 行：将该对象添加到 ObjectsFolder 下，建立 Organizes 关系。
7. 第 26-31 行：创建一个 BaseDataVariableState 类型的变量节点，表示温度值。
8. 第 32-33 行：将变量节点作为对象的组件添加，并注册到外部引用中。

5.2.3 注册节点管理器到服务端

在服务端主程序中注册自定义节点管理器，使节点生效：

csharp

```
server = new StandardServer();
server.AddNodeManager(new MyCustomNodeManager(server, configuration));
```

5.3 节点层级结构与优化

5.3.1 层级结构设计原则

设计服务端节点层级结构时应遵循以下原则：

- **清晰性**：层级结构应清晰反映设备和数据的逻辑关系。
- **可扩展性**：预留扩展接口，便于后期添加新设备或功能。
- **一致性**：采用统一的命名和组织方式，提升可维护性。

5.3.2 使用 Mermaid 绘制节点层级图

5.4 节点属性配置与数据绑定

5.4.1 动态数据绑定机制

OPC UA 服务端通常需要将节点值与实际设备数据绑定，可以使用以下方式实现动态更新：

csharp

```
temperatureVariable.OnReadValue = (context, node) =>
{
    double currentTemperature = ReadFromDevice(); // 从设备读取当前温度
    return currentTemperature;
};
```

参数说明：

- context：上下文信息，包含用户身份、权限等。
- node：当前读取的节点对象。
- ReadFromDevice()：自定义方法，模拟从设备读取数据。

5.4.2 定期刷新机制

通过定时器定期刷新变量值：

csharp

```
var timer = new Timer(RefreshTemperature, temperatureVariable, 0, 1000); // 每  
秒刷新一次
```

```
private void RefreshTemperature(object state)
{
    var variable = (BaseDataVariableState)state;
    double newValue = ReadFromDevice();
    variable.Value = newValue;
}
```

5.5 实际应用场景与优化建议

5.5.1 应用于工业物联网（IIoT）

在 IIoT 场景中，OPC UA 服务端常用于将 PLC、传感器等设备的数据统一接入平台。通过定义清晰的节点结构，可以实现设备状态的实时监控与远程控制。

5.5.2 性能优化建议

- 批量读写操作：**避免频繁调用单个节点的读写接口，应尽量使用批量处理。
- 数据缓存机制：**对于频繁读取的节点，可引入缓存机制减少设备访问次数。
- 异步处理：**对耗时操作（如设备通信）应使用异步方式，避免阻塞主线程。

6. OPC UA 服务端服务接口实现

在工业自动化系统中，OPC UA 服务端不仅是数据的提供者，更是功能接口的实现者。本章将深入探讨 OPC UA 服务端如何通过标准服务接口和自定义扩展接口，实现对客户端请求的响应，以及如何将这些接口集成到服务器地址空间中。我们将从服务端接口的组成结构入手，逐步过渡到数据访问服务、自定义方法的实现与注册流程。

6.1 服务端接口与 OPC UA 服务模型

OPC UA 服务端本质上是一个基于服务模型的系统，其核心是通过接口来定义客户端可调用的功能。服务端接口的设计直接决定了客户端如何与服务端交互。

6.1.1 OPC UA 服务端接口的组成结构

OPC UA 服务端接口主要有以下几个部分组成：

组成部分	描述
地址空间（Address Space）	包含节点及其属性，如变量、方法、对象等，是客户端访问的核心数据结构
服务接口（Service Interface）	定义了客户端可调用的服务，如读写、订阅、方法调用等
会话管理（Session Management）	处理客户端连接、身份验证、会话生命周期管理

组成部分	描述
安全策略（Security Policy）	确保通信过程中的数据完整性与机密性

OPC UA 的服务模型是基于面向对象的设计思想，每个服务接口都对应一组方法调用。例如，读写服务对应 `Read` 和 `Write` 方法，订阅服务对应 `CreateSubscription` 和 `Publish` 方法等。

6.1.2 标准服务接口与自定义服务扩展

OPC UA 协议中定义了多个标准服务接口，包括：

- 数据访问服务（Data Access Service）
- 订阅服务（Subscription Service）
- 方法调用服务（Method Call Service）

这些服务接口通常由 OPC UA SDK 提供基础实现，开发者可以在其基础上进行扩展。

此外，OPC UA 支持通过自定义服务接口来实现特定业务逻辑，例如，在一个工业控制系统中，我们可以定义一个 `StartProductionLine` 的方法，供客户端远程调用以启动生产线。

6.2 数据访问服务接口实现

数据访问服务是 OPC UA 服务端最基础也是最常用的功能之一，主要包括读取和写入服务。

6.2.1 实现读写服务接口

在 C# 中，OPC UA 服务端的数据访问服务通常通过 `StandardServer` 类和 `IServerInternal` 接口实现，可以通过继承这些类并重写其方法来定制读写逻辑。

csharp

```
// 定义一个自定义服务器类，继承自标准服务器类StandardServer
public class CustomServer : StandardServer
{
    // 重写父类的OnReadRequest方法，处理读取请求
    protected override void OnReadRequest(IServiceContext context, ReadRequest request)
    {
        // 创建读取响应对象
        var response = new ReadResponse();
        // 遍历所有需要读取的节点
        foreach (var nodeToRead in request.NodesToRead)
        {
            // 获取节点的值
            var value = GetNodeValue(nodeToRead.NodeId);
```



```

        // 将获取到的值添加到响应结果中
        response.Results.Add(new DataValue(value));
    }
    // 发送响应给请求方
    context.SendResponse(request, response);
}

// 重写父类的OnWriteRequest方法，处理写入请求
protected override void OnWriteRequest(IServiceContext context,
WriteRequest request)
{
    // 创建写入响应对象
    var response = new WriteResponse();
    // 遍历所有需要写入的节点
    foreach (var nodeToWrite in request.NodesToWrite)
    {
        // 设置节点的值
        SetNodeValue(nodeToWrite.NodeId, nodeToWrite.Value);
        // 添加操作状态码(Good表示成功)到响应结果中
        response.Results.Add(StatusCodes.Good);
    }
    // 发送响应给请求方
    context.SendResponse(request, response);
}

// 私有方法：获取指定节点的值
private object GetNodeValue(NodeId nodeId)
{
    // 实际的读取逻辑实现
    // 这里返回一个包含"CustomValue"的Variant对象作为示例
    return new Variant("CustomValue");
}

// 私有方法：设置指定节点的值
private void SetNodeValue(NodeId nodeId, DataValue value)
{
    // 实际的写入逻辑实现
    // 这里仅在控制台输出设置信息作为示例
    Console.WriteLine($"Node {nodeId} set to {value.Value}");
}
}

```

代码解释：

- OnReadRequest 和 OnWriteRequest 是 OPC UA SDK 提供的标准方法，用于处理客户端的读写请求。

- ReadRequest 和 WriteRequest 分别封装了客户端发起的读写请求数据。
- context.SendResponse 用于将结果返回给客户端。
- GetNodeValue 和 SetNodeValue 是自定义的数据访问逻辑，可以与数据库、PLC、传感器等实际数据源对接。

逻辑流程图：

6.2.2 实现订阅服务接口

订阅服务允许客户端监听节点值的变化，并通过回调机制获取数据更新。

实现步骤：

1. 创建订阅对象：

csharp

```
var subscription = new Subscription();
```

2. 设置采样间隔和触发条件：

csharp

```
subscription.PublishingInterval = 1000; // 每秒更新一次  
subscription.Triggering = Triggering.Both; // 数据变化或状态变化时触发
```

3. 添加监视项：

csharp

```
var monitoredItem = new MonitoredItem  
{  
    StartNodeId = new NodeId("MyVariable", 2),  
    AttributeId = Attributes.Value,  
    SamplingInterval = 500  
};  
subscription.MonitoredItems.Add(monitoredItem);
```

4. 注册回调函数：

csharp

```
monitoredItem.Notification += (sender, e) =>
{
    Console.WriteLine($"Node value changed: {e.Value.Value}");
};
```

5. 启动订阅服务：

csharp

```
server.Subscribe(subscription);
```

代码解释：

- Subscription 类用于定义订阅对象的基本参数。
- PublishingInterval 表示客户端接收数据更新的时间间隔。
- MonitoredItem 是订阅的具体数据项，可以设置采样间隔和触发类型。
- Notification 事件用于处理数据变化通知。

数据变化流程图：

6.3 自定义方法与服务接口注册

除了标准服务接口外，OPC UA 还支持自定义方法接口的定义与注册，使得服务端能够提供更加灵活的功能调用接口。

6.3.1 方法节点的定义与调用

在 OPC UA 中，方法节点是一种特殊类型的节点，它可以通过客户端调用执行特定的逻辑。

定义方法节点：

csharp

```
public class CustomMethodNode : MethodNode
{
    public override void OnCall(IServiceContext context, CallRequest request)
    {
        // 方法调用逻辑
        var inputArguments = request.InputArguments;
        var outputArguments = new object[] { "Method executed successfully" };

        var response = new CallResponse
        {
            OutputArguments = outputArguments
        }
    }
}
```

```

        };

        context.SendResponse(request, response);
    }
}

```

注册方法节点到地址空间：

csharp

```

var methodNode = new CustomMethodNode
{
    NodeId = new NodeId("CustomMethod", 2),
    BrowseName = new QualifiedName("CustomMethod", 2),
    DisplayName = new LocalizedText("en", "Custom Method")
};

addressSpace.AddNode(methodNode);

```

代码解释：

- CustomMethodNode 是一个自定义的方法节点类，继承自 MethodNode。
- OnCall 方法是方法节点被调用时执行的逻辑。
- CallRequest 包含客户端传入的输入参数。
- CallResponse 用于返回输出参数和调用结果。

6.3.2 将自定义服务注册至 OPC UA 服务器

要将自定义服务注册到 OPC UA 服务器中，需实现 IServiceOperation 接口，并通过 ServerManager 注册。

csharp

```

// 定义一个自定义服务操作类，实现IServiceOperation接口
public class CustomServiceOperation : IServiceOperation
{
    // 实现接口属性，返回服务操作的名称"CustomService"
    public string OperationName => "CustomService";

    // 实现接口方法，处理服务调用
    // 参数context: 服务上下文，包含请求和响应相关的信息
    // 参数request: 请求对象
    public void Invoke(IServiceContext context, object request)
    {

```

```

// 将请求对象转换为自定义请求类型CustomRequest
var customRequest = request as CustomRequest;

// 创建自定义响应对象，并设置处理结果
var response = new CustomResponse
{
    Result = $"Processed: {customRequest.Data}" // 处理请求数据并生成结果
};

// 通过上下文发送响应，将结果返回给调用方
context.SendResponse(request, response);
}
}

// 将自定义服务操作注册到服务器中，使服务器能够处理该服务请求
server.ServerManager.RegisterServiceOperation(new CustomServiceOperation());

```

代码说明：

- CustomServiceOperation 实现了 IServiceOperation 接口，定义了服务名称和调用逻辑。
- Invoke 方法用于处理客户端发送的请求。
- RegisterServiceOperation 将自定义服务注册到服务器中，供客户端调用。

自定义服务调用流程图：

7. OPC UA 服务器启动与监听

OPC UA 服务器作为数据提供方，其启动与监听机制是整个通信系统的核心。通过 C# 实现 OPC UA 服务器的初始化、启动流程、运行时监听逻辑以及异常处理机制，确保服务器稳定运行并有效响应客户端请求。

7.1 OPC UA 服务器的启动流程

7.1.1 配置服务器配置文件与端点信息

OPC UA 服务器的启动依赖于配置文件的正确设置。通常，服务器的配置信息包含端点地址、端口号、安全策略、证书路径等。

接下来是一个典型的服务器配置示例（serverconfig.json）：

json

```
{
  "ServerName": "MyOPCUAServer",
  "Port": 51210,
  "BaseAddress": "opc.tcp://localhost:51210",
  "SecurityPolicies": [
    "http://opcfoundation.org/UA/SecurityPolicy#None",
    "http://opcfoundation.org/UA/SecurityPolicy#Basic256"
  ],
  "CertificatePath": "./certificates/server.pfx",
  "CertificatePassword": "password123"
}
```

在 C# 中，可以通过读取配置文件并初始化服务器配置对象：

csharp

```
// 从"serverconfig.json"文件中读取所有文本内容，并将其反序列化为ServerConfiguration对象
// 这行代码的作用是加载JSON配置文件并转换为C#可操作的对象
var config = JsonConvert.DeserializeObject<ServerConfiguration>(
  File.ReadAllText("serverconfig.json"));

// 创建一个新的ServerConfiguration对象，用于实际配置服务器
var serverConfiguration = new ServerConfiguration
{
  // 设置服务器名称，值来自于前面加载的配置对象
  ServerName = config.ServerName,
  // 设置服务器基础地址列表，将配置中的单个地址放入列表中
  BaseAddresses = new List<string> { config.BaseAddress },
  // 设置安全策略列表，将配置中的安全策略转换为List集合
  SecurityPolicies = config.SecurityPolicies.ToList(),
  // 设置证书文件路径，值来自于配置对象
  CertificateFile = config.CertificatePath,
  // 设置证书密码，值来自于配置对象
  CertificatePassword = config.CertificatePassword
};
```

7.1.2 初始化地址空间与服务端节点

在服务器启动前，需要完成地址空间的构建，包括添加自定义节点、绑定变量等。通常，地址空间初始化由 `ServerManager` 或自定义的地址空间管理类完成。

csharp

```
// 简单的地址空间初始化示例
// 创建地址空间对象，用于存储OPC UA服务器中的所有节点信息
var addressSpace = new AddressSpace();
// 创建自定义节点管理器，用于管理地址空间中的节点，参数为上面创建的地址空间
var nodeManager = new CustomNodeManager(addressSpace);

// 添加一个模拟变量节点
// 创建基础数据变量节点，父节点为根文件夹(通过节点管理器获取)
var variableNode = new BaseDataVariableState(nodeManager.GetRootFolder())
{
    // 设置节点ID，标识符为"MyVariable"，命名空间索引为2
    NodeId = new NodeId("MyVariable", 2),
    // 设置浏览名称，名称为"MyVariable"，命名空间索引为2
    BrowseName = new QualifiedName("MyVariable", 2),
    // 设置显示名称，语言为英文，显示文本为"MyVariable"
    DisplayName = new LocalizedText("en", "MyVariable"),
    // 设置数据类型为双精度浮点数(引用OPC UA标准数据类型ID)
    DataType = DataTypeIds.Double,
    // 设置初始值为0.0
    Value = 0.0
};

// 将创建的变量节点添加到地址空间中，使其在服务器中可见并可访问
addressSpace.AddNode(variableNode);
```

7.2 实现服务器运行时监听机制

7.2.1 接收客户端连接请求与会话管理

OPC UA 服务器启动后，需持续监听来自客户端的连接请求，并维护客户端会话状态。会话管理包括会话创建、心跳检测、会话保持与释放等。

7.2.2 监听数据访问与订阅请求

服务器需要监听客户端的数据访问请求（如读写节点）和订阅请求（如创建订阅、添加监控项），这些请求由服务器内置的服务接口处理，开发者可自定义实现回调逻辑。

例如，当客户端请求读取节点值时，可以实现以下回调逻辑：

csharp

```
server.DataRead += (sender, e) =>
{
    Console.WriteLine($"Data read request for node: {e.NodeId}");
    if (e.NodeId.Identifier.ToString() == "MyVariable")
```

```
{
    e.Value = addressSpace.GetVariableValue(e.NodeId);
}
};
```

对于订阅请求的监听，开发者可订阅 `SubscriptionCreated` 和 `MonitoredItemCreated` 事件：

csharp

```
server.SubscriptionCreated += (sender, e) =>
{
    Console.WriteLine($"Subscription created: {e.SubscriptionId}");
};

server.MonitoredItemCreated += (sender, e) =>
{
    Console.WriteLine($"Monitored item added: {e.ItemToMonitor.NodeId}");
};
```

7.3 服务器异常处理与日志记录

7.3.1 异常捕获与服务器稳定性保障

在服务器运行过程中，必须对可能出现的异常进行捕获和处理，以确保服务器的稳定性。常见的异常类型包括网络中断、客户端异常断开、地址空间访问越界等。

全局异常处理示例：

csharp

```
// 注册应用程序域的未处理异常事件
// 当应用程序中发生未被捕获的异常时触发
// sender: 事件发送者（通常是当前应用程序域）
// args: 事件参数，包含未处理的异常信息
// 这个事件可以捕获大多数应用程序级别的未处理异常
AppDomain.CurrentDomain.UnhandledException += (sender, args) =>
{
    // 将异常对象转换为Exception类型
    var exception = (Exception)args.ExceptionObject;
    // 输出未处理异常的错误信息到标准错误流
    Console.Error.WriteLine($"未处理的异常: {exception.Message}");
    // 这里可以添加日志记录逻辑
    // 也可以实现服务重启等恢复机制
};
```



```
// 注册未观察到的任务异常事件
// 当Task中发生的异常未被观察和处理时触发
// 这对于捕获异步操作中遗漏的异常非常重要
TaskScheduler.UnobservedTaskException += (sender, args) =>
{
    // 输出未观察到的任务异常信息
    Console.Error.WriteLine($"未观察到的任务异常: {args.Exception.Message}");
    // 标记异常为已观察，防止应用程序因此终止
    // 在.NET 4.5及以上版本，如果不标记，可能导致进程终止
    args.SetObserved();
};
```

7.3.2 日志记录与远程调试支持

为了便于调试和问题排查，服务器应具备完善的日志记录机制。可使用 NLog 或 Serilog 等日志框架记录服务器运行状态。

简单日志记录配置实例：

csharp

```
using System;
using NLog;

class NLogExample
{
    // 获取日志记录器实例
    private static readonly Logger logger =
    LogManager.GetCurrentClassLogger();

    static void Main(string[] args)
    {
        try
        {
            // 记录不同级别的日志
            logger.Trace("这是一条跟踪日志 - 最详细的调试信息");
            logger.Debug("这是一条调试日志 - 用于开发调试");
            logger.Info("这是一条信息日志 - 记录正常操作");
            logger.Warn("这是一条警告日志 - 表示潜在问题");
            logger.Error("这是一条错误日志 - 表示操作失败");
            logger.Fatal("这是一条致命错误日志 - 表示严重系统错误");

            // 演示异常日志
            throw new InvalidOperationException("演示异常情况");
        }
        catch (Exception ex)
```

```

    {
        // 记录异常信息
        logger.Error(ex, "发生了一个异常");
    }
    finally
    {
        // 确保日志被刷新并关闭
        LogManager.Shutdown();
    }
}
}

```

xml

```

<?xml version="1.0" encoding="utf-8" ?>
<nlog xmlns="http://www.nlog-project.org/schemas/NLog.xsd"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      autoReload="true"
      internalLogLevel="Info"
      internalLogFile="internal-nlog.log">

  <!-- 目标：日志输出的位置 -->
  <targets>
    <!-- 控制台输出 -->
    <target xsi:type="Console" name="console"
            layout="${longdate} ${level:uppercase=true} ${logger} -
${message} ${exception}" />

    <!-- 文件输出 -->
    <target xsi:type="File" name="file"
            fileName="logs/${shortdate}.log"
            layout="${longdate} ${level:uppercase=true} ${logger} -
${message} ${exception}"
            archiveEvery="Day" <!-- 每天归档 -->
            archiveNumbering="Rolling" <!-- 滚动编号 -->
            maxArchiveFiles="7" /> <!-- 最多保留7个归档文件 -->
  </targets>

  <!-- 规则：哪些日志被发送到哪些目标 -->
  <rules>
    <!-- 将所有级别的日志发送到控制台 -->
    <logger name="*" minlevel="Trace" writeTo="console" />

    <!-- 将Info及以上级别的日志发送到文件 -->
    <logger name="*" minlevel="Info" writeTo="file" />
  </rules>
</nlog>

```

```
</rules>  
</nlog>
```

此外，可集成远程调试模块，通过 OPC UA 节点暴露日志信息或服务器状态，供客户端远程访问查看。